

Sistema de Control Seguro para Brazo Robótico Pick & Place mediante MQTT, FastAPI y JWT

Francisco Norberto Bevilacqua¹

¹ Ingeniería en Computación II, Facultad de Tecnologías e Innovación para el Desarrollo,
Bv. Roca 989, 2300 Rafaela, Argentina — bevilacquafrancisco1@gmail.com
<https://www.unraf.edu.ar/>

Resumen: El control remoto de sistemas robóticos en entornos industriales presenta desafíos técnicos significativos en materia de comunicación en tiempo real, autenticación de operadores y seguridad de acceso. Este trabajo describe el diseño, implementación y verificación de un sistema de control seguro para un brazo robótico de cuatro grados de libertad del tipo pick & place, desarrollado como proyecto integrador de la asignatura Ingeniería en Computación II. La solución migró desde un broker MQTT público y sin autenticación hacia una arquitectura en tres capas compuesta por: un broker Mosquitto privado con listas de control de acceso (ACL) por usuario, un backend de autenticación construido con FastAPI que emite tokens JWT mediante hashing bcrypt, y una interfaz gráfica web que requiere inicio de sesión previo al acceso al panel de control. El firmware del microcontrolador ESP32, desarrollado en MicroPython, implementa tres modos de operación (manual, semiautomático y automático), comunicación MQTT con garantía de entrega QoS 1, deduplicación de mensajes y reconexión automática. La arquitectura fue diseñada aplicando principios de separación de responsabilidades, mínimo privilegio y defensa en profundidad. Los resultados demuestran un sistema completamente funcional con autenticación de dos capas independientes, control de acceso a tópicos MQTT y operación estable en red de área local.

Palabras claves: MQTT, ESP32, FastAPI, JWT, Pick & Place, Ciberseguridad IoT, MicroPython.

1. Introducción

La automatización industrial basada en microcontroladores conectados a redes de comunicación locales constituye uno de los paradigmas centrales de la Industria 4.0. Los sistemas robóticos de clasificación y transporte de piezas, como los brazos del tipo pick & place, son ampliamente utilizados en líneas de producción, y su gestión remota a través de interfaces web representa una mejora sustancial respecto del control local por panel físico.

Sin embargo, la exposición de estos sistemas a redes de comunicación introduce vectores de ataque que deben considerarse desde las primeras etapas del diseño. El uso de brokers MQTT públicos sin autenticación, la ausencia de control de acceso a tópicos y la falta de verificación de identidad del operador representan vulnerabilidades concretas identificadas en proyectos previos de la asignatura [1, 2].

El presente trabajo aborda estas problemáticas mediante el desarrollo de un sistema de control en tres capas que integra: (1) un broker MQTT privado con autenticación por credenciales y ACL por usuario, (2) un backend REST de autenticación con emisión de tokens JWT, y (3) una interfaz gráfica web que requiere sesión autenticada antes de habilitar el panel de control del brazo robótico.

El objetivo general es diseñar e implementar un sistema de control para brazo robótico pick & place que cumpla con los principios de mínimo privilegio, separación de responsabilidades y defensa en profundidad, siendo funcional en una red de área local de laboratorio. Los objetivos específicos

son: (a) migrar la comunicación desde un broker público a uno privado con autenticación; (b) implementar autenticación de operadores mediante JWT con hashing bcrypt; (c) desarrollar una interfaz gráfica web con pantalla de login integrada; (d) verificar el sistema mediante pruebas funcionales documentadas.

El presente trabajo se organiza de la siguiente forma: en la sección 2 se revisa el estado del arte; en la sección 3 se presenta el marco teórico; en la sección 4 se analiza el problema y los requerimientos; en la sección 5 se describe la metodología y la arquitectura de la solución; en la sección 6 se detalla la implementación; en la sección 7 se presentan las pruebas, resultados y discusión; y en la sección 8 se exponen las conclusiones.

2. Estado del Arte

El protocolo MQTT (Message Queuing Telemetry Transport) fue diseñado originalmente por IBM para telemetría en redes de baja confiabilidad [3]. Su modelo publicador-suscriptor lo convierte en el estándar de facto para comunicación IoT industrial, siendo adoptado en plataformas como AWS IoT Core, Azure IoT Hub y Mosquitto. La versión 5.0 del protocolo introduce mejoras como identificadores de razón de desconexión y propiedades de usuario, aunque la versión 3.1.1 sigue siendo dominante en proyectos académicos y sistemas embebidos con recursos limitados [4].

En el ámbito del control robótico sobre redes, existen propuestas que combinan ESP32 con comunicación MQTT para robots móviles [5] y brazos manipuladores [6]. Sin embargo, la mayoría de estas implementaciones utilizan brokers públicos sin autenticación (como test.mosquitto.org), lo que expone los comandos enviados al sistema a cualquier cliente conectado al mismo broker.

En materia de seguridad IoT, el proyecto OWASP IoT Top 10 [7] identifica como principales vulnerabilidades la ausencia de mecanismos de autenticación, el uso de contraseñas débiles o predeterminadas y la falta de cifrado en el transporte. La implementación de listas de control de acceso (ACL) a nivel de broker y la autenticación de operadores mediante tokens JWT representa una aplicación directa de estos lineamientos en el contexto de un sistema embebido académico.

FastAPI [8] es un framework Python moderno que genera documentación interactiva automáticamente mediante OpenAPI, lo que facilita tanto la verificación de endpoints durante el desarrollo como la presentación del sistema en defensa académica. Su integración con Pydantic permite validación de datos estricta desde la capa de entrada, reduciendo la superficie de ataque ante entradas malformadas.

La brecha identificada respecto de trabajos similares es la ausencia de una arquitectura que combine autenticación en dos capas independientes: una a nivel de transporte (broker MQTT con ACL) y otra a nivel de aplicación (JWT para operadores humanos), aplicada a un sistema robótico con múltiples modos de operación y panel de control web.

3. Marco Teórico

3.1 Protocolo MQTT y modelo publicador-suscriptor

MQTT es un protocolo de mensajería liviano basado en el modelo publicador-suscriptor. Los clientes no se comunican directamente entre sí, sino a través de un broker central que recibe mensajes publicados en tópicos y los distribuye a los suscriptores de esos tópicos. Este desacoplamiento permite que el ESP32 publique telemetría en robot/log sin conocer la cantidad ni la identidad de los clientes que la consumen, y que la GUI publique comandos en robot/cmd sin conocer la implementación del firmware.

El nivel de calidad de servicio (QoS) determina las garantías de entrega. QoS 0 es fire-and-forget; QoS 1 garantiza entrega al menos una vez mediante acuse de recibo (PUBACK), lo que puede producir duplicados; QoS 2 garantiza exactamente una entrega. El firmware implementa QoS 1 en

la suscripción a robot/cmd y aplica deduplicación mediante un campo msg_id único en el payload JSON, logrando efectivamente semántica de exactamente una vez a nivel de aplicación.

3.2 Autenticación mediante JSON Web Tokens (JWT)

Un JWT es un token compacto y autocontenido que representa afirmaciones entre dos partes, firmado digitalmente [9]. Su estructura es header.payload.signature, donde la firma se calcula con HMAC-SHA256 sobre los primeros dos segmentos usando un secreto compartido. El backend verifica la firma sin necesidad de consultar una base de datos, lo que hace al mecanismo stateless y escalable. El claim exp (expiration) establece el vencimiento del token, y el claim sub (subject) identifica al operador autenticado.

3.3 Hashing de contraseñas con bcrypt

bcrypt es una función de derivación de clave diseñada para el almacenamiento de contraseñas [10]. A diferencia de SHA-256, bcrypt incluye un factor de costo configurable (cost factor) que incrementa el tiempo de cómputo de forma deliberada, haciendo los ataques de fuerza bruta computacionalmente costosos. Incluye un salt aleatorio por hash, lo que garantiza que dos contraseñas idénticas produzcan hashes distintos, mitigando ataques de tabla precomputada (rainbow tables). El hash resultante tiene el formato \$2b\$12\$salt22chars.hashchars, donde 12 es el número de rondas.

3.4 Control de acceso a tópicos MQTT mediante ACL

Mosquitto implementa listas de control de acceso (ACL) que definen, por usuario, qué tópicos puede publicar y a cuáles puede suscribirse. Las directivas topic read y topic write permiten especificar permisos de forma granular. La directiva allow_anonymous false obliga a que todo cliente presente credenciales válidas antes de conectarse. Esta combinación aplica el principio de mínimo privilegio: el ESP32 solo puede publicar en robot/log y suscribirse a robot/cmd, mientras que la GUI tiene los permisos inversos.

3.5 ESP32 y MicroPython

El ESP32 es un microcontrolador de doble núcleo con WiFi y Bluetooth integrados, fabricado por Espressif Systems. MicroPython es una implementación del intérprete Python 3 optimizada para microcontroladores con recursos limitados. La librería umqtt.simple provee una implementación minimalista del cliente MQTT que soporta QoS 0 y QoS 1, autenticación con usuario y contraseña, y keepalive configurable. El control de servomotores SG90 se realiza mediante señales PWM a 50 Hz con ciclos de trabajo entre 0.5 ms y 2.5 ms correspondientes a posiciones de 0° a 180°.

3.6 Principios de seguridad aplicados

El diseño aplica tres principios del framework OWASP: (1) mínimo privilegio, restringiendo los permisos de cada componente al mínimo necesario; (2) defensa en profundidad, implementando autenticación en dos capas independientes (MQTT y JWT); y (3) fail-fast, haciendo que el backend no arranque si alguna variable de configuración crítica está ausente, en lugar de continuar en un estado parcialmente configurado.

4. Análisis del Problema y Requerimientos

4.1 Descripción del problema y alcance

El sistema previo (v4.0) conectaba el ESP32 al broker público test.mosquitto.org sin autenticación. Cualquier cliente con acceso a Internet podía publicar en robot/cmd y enviar comandos al brazo robótico, o suscribirse a robot/log y recibir telemetría del sistema. Adicionalmente, el panel de control web era accesible sin verificación de identidad del operador.

El alcance del proyecto contempla: la migración a un broker Mosquitto privado en red local, la implementación de autenticación de operadores con JWT, la integración de una pantalla de login

en la interfaz gráfica, y la verificación funcional completa del sistema integrado. Queda fuera del alcance la implementación de TLS (cifrado en tránsito), refresh tokens y gestión dinámica de usuarios, elementos documentados como trabajo futuro.

4.2 Requerimientos funcionales

ID	Requerimiento Funcional
RF1	El broker MQTT debe rechazar conexiones de clientes sin credenciales válidas
RF2	El usuario esp32 solo puede publicar en robot/log y suscribirse a robot/cmd
RF3	El usuario gui_operator solo puede publicar en robot/cmd y suscribirse a robot/log
RF4	El backend debe emitir un JWT firmado tras verificar usuario y contraseña con hash bcrypt
RF5	La GUI debe redirigir a login.html si no existe sesión JWT válida al cargar index.html
RF6	El firmware debe operar en tres modos: MANUAL, SEMI_AUTO y AUTOMATICO
RF7	El modo MANUAL permite control directo de cada servo mediante sliders y acciones rápidas
RF8	El modo SEMI_AUTO detecta cajas vía sensor KY-032 y espera decisión del operador
RF9	El modo AUTOMATICO ejecuta secuencias completas de pick & place sin intervención humana
RF10	El firmware debe reconectarse automáticamente al broker ante caídas de red
RF11	El backend debe bloquear IPs que superen 5 intentos de login fallidos en 60 segundos
RF12	La GUI debe mostrar indicador de estado del backend antes de intentar autenticación

Tabla 1. Requerimientos funcionales del sistema.

4.3 Requerimientos no funcionales

ID	Requerimiento No Funcional	Categoría
RNF1	Las contraseñas de operadores deben almacenarse con hash bcrypt, nunca en texto plano	Seguridad
RNF2	El JWT debe expirar en 60 minutos como máximo	Seguridad
RNF3	El secreto JWT debe generarse con mínimo 32 bytes de entropía (secrets.token_hex)	Seguridad
RNF4	El broker debe aplicar ACL que restrinja el acceso a tópicos por usuario	Seguridad
RNF5	Los mensajes de error del login no deben distinguir "usuario no existe" de "contraseña incorrecta"	Seguridad
RNF6	El backend debe arrancar con fallo explícito si falta alguna variable de entorno crítica	Confiabilidad

RNF7	La latencia de respuesta del brazo robótico ante un comando debe ser menor a 500 ms	Rendimiento
RNF8	El sistema debe operar de forma estable en red WiFi de área local (LAN)	Portabilidad
RNF9	La GUI debe ser accesible desde cualquier navegador moderno sin instalación de plugins	Usabilidad

Tabla 2. Requerimientos no funcionales del sistema.

5. Metodología y Arquitectura de la Solución

5.1 Enfoque metodológico

El desarrollo siguió un proceso iterativo-incremental dividido en tres fases explícitas, cada una con sus propias decisiones de arquitectura (ADR), implementación y verificación antes de avanzar a la siguiente:

- Fase 1: Migración del broker MQTT de público a privado, configuración de autenticación y ACL, verificación con cuatro pruebas cruzadas (autenticación válida, rechazo sin credenciales, rechazo de ACL cruzada, WebSocket).
- Fase 2: Implementación del backend FastAPI con autenticación JWT, hashing bcrypt, rate limiting y validación de configuración fail-fast.
- Fase 3: Integración de pantalla de login en la GUI, módulo auth.js autocontenido, guard de sesión antes del pintado del panel de control.

Cada ADR documenta el contexto, las opciones consideradas, la decisión tomada y las consecuencias aceptadas. El control de versiones se gestionó con Git, con commits descriptivos al cierre de cada fase.

5.2 Visión general de la arquitectura

La arquitectura del sistema se organiza en cuatro componentes principales que interactúan a través de dos protocolos de comunicación independientes:

- ESP32 (firmware MicroPython): publica eventos en robot/log y se suscribe a robot/cmd a través de MQTT sobre TCP (puerto 1883).
- Broker Mosquitto privado: intermediario central que gestiona la distribución de mensajes, impone autenticación y aplica ACL por usuario. Expone también un listener WebSocket en el puerto 9001 para la GUI.
- Backend FastAPI: expone los endpoints POST /auth/login y GET /auth/verify. No interactúa con el broker MQTT; su única responsabilidad es autenticar operadores humanos y emitir JWT.
- GUI Web (navegador): se conecta al broker via WebSocket Paho MQTT y al backend via fetch HTTP. La sesión JWT se almacena en sessionStorage y se verifica antes de habilitar el panel de control.

La separación entre la autenticación MQTT (credenciales de la aplicación ante el broker) y la autenticación JWT (identidad del operador humano ante el backend) es una decisión de diseño deliberada que aplica el principio de defensa en profundidad: un compromiso de uno de los dos mecanismos no anula automáticamente al otro.

5.3 Capa de presentación — GUI Web

La interfaz gráfica está construida con HTML, CSS y JavaScript vanilla sin frameworks adicionales, lo que elimina dependencias de build y permite servir los archivos directamente con cualquier

servidor HTTP estático. La librería Paho MQTT sobre WebSocket provee la comunicación con el broker desde el navegador.

El módulo `auth.js` es el único componente de la GUI que conoce el backend FastAPI y el formato JWT. Expone una API pública de cinco funciones: `attemptLogin`, `verifySessionRemote`, `requireSession`, `logout` y `getOperatorName`. El script `robot_script.js` consume estas funciones sin conocer su implementación, lo que aplica el Principio de Responsabilidad Única (SRP) entre módulos.

La pantalla de login (`login.html`) incluye un indicador de estado del backend en tiempo real que consulta el endpoint `GET /health` antes de que el operador intente autenticarse. Esto convierte el problema habitual de "el backend está caído" de un error confuso en información visible y accionable.

El panel principal implementa cuatro badges de estado (MQTT, ESP32, Sensor, Modo) que se actualizan en tiempo real a partir de los eventos recibidos del firmware. La consola de logs categoriza los mensajes por nivel (DEBUG, INFO, WARN, ERROR) y por dirección (← ESP32, → GUI, Sistema), con filtrado visual por nivel.

5.4 Capa de lógica — Backend FastAPI

El backend sigue una arquitectura de capas con separación estricta de responsabilidades. El módulo `config.py` es el único punto de acceso a variables de entorno; ningún otro módulo lee `os.environ` directamente. El módulo `security.py` concentra toda la lógica criptográfica (`bcrypt` y JWT). El módulo `rate_limit.py` implementa el limitador de intentos de login en memoria de proceso. El módulo `dependencies.py` expone la dependencia `get_current_username` que protege endpoints con JWT.

Esta organización garantiza que una eventual auditoría de seguridad solo necesite revisar `security.py` para verificar toda la superficie criptográfica del sistema. El router `auth.py` orquesta estos módulos pero no implementa criptografía directamente, aplicando el principio de inversión de dependencias (DIP de SOLID).

5.5 Capa de comunicación — Broker Mosquitto

El broker Mosquitto expone dos listeners con la misma configuración de autenticación: TCP en el puerto 1883 para el ESP32 (que no puede usar WebSocket por limitaciones de `umqtt.simple`) y WebSocket en el puerto 9001 para la GUI (cuyo navegador no puede abrir conexiones TCP directas). La directiva `allow_anonymous false` garantiza que todo cliente debe presentar credenciales antes de conectarse.

El archivo `acl.conf` define permisos mínimos por usuario. Esta separación entre autenticación (`passwd`) y autorización (`acl.conf`) permite gestionar cada dimensión de seguridad de forma independiente, lo que facilita futuras extensiones como agregar usuarios con distintos niveles de acceso.

5.6 Firmware — ESP32 MicroPython

El firmware implementa un loop principal no bloqueante basado en diferencias de timestamps (`ticks_ms`) que gestiona en paralelo: polling MQTT, lectura del sensor, heartbeat de telemetría y garbage collector. Este diseño evita llamadas a `sleep()` que bloquearían la atención al broker y al watchdog.

Los tres modos de operación son gestionados por una máquina de estados implícita controlada por la variable global `mode`. En modo MANUAL, los comandos de servo se ejecutan inmediatamente. En modo SEMI_AUTO, la detección del sensor activa una bandera `semi_pending` y el firmware espera el comando `semi_decision` del operador antes de ejecutar la secuencia. En modo AUTOMATICO, el firmware toma la decisión de destino sin intervención humana, priorizando Pallet 1 sobre Pallet 2.

6. Implementación

6.1 Tecnologías y decisiones de diseño

Las decisiones de tecnología se documentaron como ADR antes de implementar. Las más relevantes son:

- FastAPI sobre Django: para un backend de autenticación stateless con dos endpoints, Django es sobredimensionado. FastAPI genera documentación interactiva automática (/docs) sin configuración adicional, lo que facilita la verificación durante el desarrollo y la presentación en defensa.
- JWT sin refresh token: el sistema opera en sesiones de demo de duración acotada. Agregar refresh tokens introduciría complejidad (almacenamiento de tokens, revocación) sin beneficio práctico en este alcance. Documentado como trabajo futuro para operación continua.
- Usuarios en .env, no en base de datos: dos usuarios estáticos no justifican la complejidad de una BD. Si el sistema escalara a gestión dinámica de operadores, esta capa migraría a un ROL 9 (DBA) sin tocar la lógica de autenticación.
- sessionStorage sobre localStorage para el JWT: sessionStorage se borra al cerrar la pestaña, reduciendo la ventana de exposición del token si la PC de operación queda desatendida. No hay necesidad de persistencia entre sesiones para este caso de uso.
- Mosquitto 2.0.21 sobre 2.1.x: la versión 2.1.2 presentó incompatibilidades en Windows con el listener WebSocket cuando se usa la directiva per_listener_settings, resultando en un proceso que arrancaba pero no inicializaba el puerto 9001. La versión 2.0.21 opera establemente con la configuración global de allow_anonymous.

6.2 Estructura del proyecto

El proyecto se organiza en cuatro subdirectorios con responsabilidades delimitadas:

```
TP-IC2/
├── docs/
│   └── BevilacquaFrancisco_IC-II.docx    # Informe técnico completo de la
cátedra
├── src/
│   ├── backend/                          # API de autenticación (FastAPI)
│   │   ├── app/
│   │   │   ├── core/                    # config, security, rate_limit,
dependencies
│   │   │   ├── routers/                 # endpoints /auth/*
│   │   │   ├── schemas/                 # modelos Pydantic de
request/response
│   │   │   └── main.py                  # entry point de la app
│   │   └── scripts/
│   │       └── generar_hash_password.py # utilidad CLI para generar hashes
bcrypt
│   └── .env.example                      # plantilla de configuración (sin
secrets)
│       ├── generar_passwd.py            # utilidad para el archivo passwd
de Mosquitto
│       └── requirements.txt
│   ├── firmware/                        # Firmware ESP32 (MicroPython,
modular)
│   └── main.py / robot_main.py          # entry point
```

```

|   |   |— config.py                    # pines, WiFi, credenciales MQTT
|   |   |— mqtt.py                      # cliente MQTT + reconexión
|   |   |— servos.py / sensor.py        # control de hardware
|   |   |— commands.py                  # router de comandos entrantes
|   |   |— state.py / wifi.py
|   |— gui/                             # Panel de control web
|   |   |— login.html / login_script.js / login_style.css
|   |   |— index.html
|   |   |— auth.js                      # sesión JWT (login, verify, guard)
|   |   |— robot_script.js              # cliente MQTT + lógica de control
|   |   |— robot_style.css
|   |— mosquitto-broker/
|   |   |— mosquitto.conf                # listeners TCP/WS, auth
obligatoria
|   |   |— acl.conf                      # control de acceso por
usuario/tópico
|   |— ARRANCAR_SISTEMA.ps1              # script de arranque para Windows
|— .gitignore
|— README.md

```

Fragmento representativo — Función de login en el backend (app/routers/auth.py):

```

if is_rate_limited(client_ip):
    raise HTTPException(status_code=429, detail="Demasiados intentos...")

stored_hash = settings.auth_users.get(body.username)
hash_to_check = stored_hash if stored_hash else _DUMMY_HASH
password_ok = verify_password(body.password, hash_to_check)

if not stored_hash or not password_ok:
    register_failed_attempt(client_ip)
    raise HTTPException(status_code=401, detail="Usuario o contraseña
incorrectos.")

```

El uso del hash dummy cuando el usuario no existe mitiga un timing attack: si el sistema retornara inmediatamente al no encontrar el usuario, un atacante podría inferir la existencia de usuarios válidos midiendo el tiempo de respuesta, ya que la verificación bcrypt tarda deliberadamente varias decenas de milisegundos.

Fragmento representativo — ACL del broker (acl.conf):

```

user esp32
topic write robot/log
topic read  robot/cmd

user gui_operator
topic read  robot/log
topic write robot/cmd

```

Fragmento representativo — Guard de sesión en la GUI (robot_script.js):


```

window.onload = async () => {
    const operatorName = await requireSession();
    document.body.classList.remove('auth-pending');
    const txtOperator = el('txt-operator');
    if (txtOperator) txtOperator.textContent = operatorName;
    connectMQTT();
};

```

La función `requireSession()` retorna una promesa que nunca resuelve si no hay sesión válida, deteniendo la ejecución del resto del `onload` antes de que `connectMQTT()` se ejecute.

6.3 Protocolo de comandos y eventos MQTT

La comunicación entre la GUI y el firmware se realiza mediante JSON sobre MQTT. Los comandos en `robot/cmd` siguen el esquema:

Comando	Campos	Descripción
set_mode	mode: MANUAL SEMI_AUTO AUTOMATICO	Cambia el modo de operación del firmware
servo	id: 1-4, angle: 0-180	Mueve un servo individual (QoS 0)
move	action: home recolectar abrir_pinza cerrar_pinza	Ejecuta movimiento preconfigurado
semi_decision	dest: P1 P2 ignorar	Respuesta a detección de caja en modo SEMI_AUTO
pallet_clear	pallet: 1 2	Informa que un pallet fue vaciado físicamente
status	(ninguno)	Solicita telemetría completa inmediata

Tabla 3. Protocolo de comandos en el tópico `robot/cmd`.

Los eventos en `robot/log` siguen el mismo esquema JSON. Los comandos de acción crítica (`set_mode`, `move`, `semi_decision`, `pallet_clear`) incluyen un campo `msg_id` único generado con `Date.now()` + número aleatorio. El firmware descarta mensajes con `msg_id` igual al último procesado, neutralizando los duplicados inherentes al protocolo QoS 1.

6.4 Modos de operación del firmware

El modo **MANUAL** permite al operador controlar individualmente cada servo mediante los sliders de la GUI (ángulo 0°–180°) y ejecutar cuatro movimientos preconfigurados: HOME (posición de reposo segura), Recolectar (mueve el brazo a la zona del sensor con pinza abierta), Abrir Pinza y Cerrar Pinza. Todos los comandos en este modo se publican con QoS 0 para minimizar latencia, ya que el valor más reciente del slider siempre reemplaza al anterior.

El modo **SEMI_AUTO** utiliza el sensor KY-032 (detector infrarrojo de obstáculos) para detectar la presencia de una caja en la zona de recolección. El sensor se lee con debounce de cinco muestras consecutivas para evitar falsas detecciones por ruido. Al confirmar la detección, el firmware publica el evento `box_detected` y activa la bandera `semi_pending`, bloqueando nuevas detecciones. La GUI habilita los tres botones de decisión (Pallet 1, Pallet 2, Ignorar) y espera la respuesta del operador. El firmware ejecuta entonces la secuencia de pick & place hacia el destino seleccionado.

El modo **AUTOMATICO** aplica la misma lógica de detección pero toma la decisión de destino sin intervención humana, priorizando Pallet 1 hasta alcanzar tres cajas, luego Pallet 2. Al llenar ambos pallets, el firmware publica `all_pallets_full` y detiene el proceso hasta recibir un comando `pallet_clear` del operador.

La secuencia de pick & place consta de nueve pasos documentados en el firmware: tránsito seguro (levantar hombro y codo antes de girar la base para evitar tumbar cajas), aproximación a la zona de recolección, cierre de pinza, tránsito hacia el pallet destino, descenso al nivel de apilado correcto, apertura de pinza, tránsito de salida, y retorno a la zona de recolección.

7. Pruebas, Resultados y Discusión

7.1 Pruebas del broker Mosquitto

ID	Caso de prueba	Req.	Resultado
CP1	Publicación con credenciales válidas: mosquitto_pub -u gui_operator -P <password> -t robot/cmd	RF1	OK — mensaje recibido en suscriptor
CP2	Conexión sin credenciales: mosquitto_sub -t robot/cmd (sin -u ni -P)	RF1	OK — rechazado: Connection Refused: not authorised
CP3	ACL cruzada: gui_operator intenta publicar en robot/log	RF2, RF3	OK — denegado en log del broker
CP4	Conexión WebSocket: mosquitto_sub -p 9001 -u gui_operator	RF1	OK — conecta y queda a la espera

Tabla 4. Pruebas de verificación del broker Mosquitto.

7.2 Pruebas del backend FastAPI

ID	Caso de prueba	Req.	Resultado
CP5	POST /auth/login con credenciales correctas	RF4	OK — JWT emitido, estructura correcta
CP6	GET /auth/verify con token válido	RF5	OK — username confirmado, valid: true
CP7	POST /auth/login con contraseña incorrecta	RNF5	OK — HTTP 401, mensaje genérico
CP8	POST /auth/login con usuario inexistente	RNF5	OK — mismo mensaje que CP7 (sin distinguir causa)
CP9	GET /auth/verify sin token en el header	RF5	OK — HTTP 403
CP10	GET /auth/verify con token manipulado	RF5	OK — HTTP 401, mensaje genérico
CP11	6 intentos fallidos consecutivos desde la misma IP	RF11	OK — 6to intento devuelve HTTP 429, incluso con contraseña correcta

Tabla 5. Pruebas de verificación del backend FastAPI.

7.3 Pruebas de integración del sistema completo

ID	Caso de prueba	Req.	Resultado
CP12	Acceso a index.html sin sesión JWT activa	RF5	OK — redirige automáticamente a login.html

CP13	Login exitoso y acceso al panel	RF4, RF5	OK — badge OPERADOR muestra el usuario, conexión MQTT establecida
CP14	Control manual de servo desde slider	RF7	OK — brazo responde en < 500 ms
CP15	Detección de caja en modo SEMI_AUTO	RF8	OK — botones de decisión se habilitan, alerta visible
CP16	Secuencia automática completa (modo AUTO)	RF9	OK — brazo ejecuta pick & place sin intervención
CP17	Logout y verificación de sesión invalidada	RF5	OK — acceso a index.html redirige a login.html
CP18	Reconexión MQTT tras caída de red simulada	RF10	OK — reconecta en < 5 s, publica evento online

Tabla 6. Pruebas de integración del sistema completo.

7.4 Discusión

Todos los requerimientos funcionales y no funcionales fueron satisfechos durante las pruebas. El sistema demostró operación estable en sesiones continuas de más de 30 minutos, con reconexiones automáticas exitosas al broker tanto desde el firmware como desde la GUI.

La comparación con el estado del arte muestra que la arquitectura desarrollada supera las implementaciones típicas revisadas en la sección 2, que utilizan brokers sin autenticación. La adición de autenticación en dos capas independientes (MQTT y JWT) representa una mejora sustancial en la postura de seguridad del sistema sin incrementar significativamente la complejidad de uso para el operador.

Las principales dificultades encontradas durante la implementación fueron de naturaleza operativa y de compatibilidad de versiones: la incompatibilidad entre Mosquitto 2.1.2 y el listener WebSocket en Windows, la incompatibilidad de formato de hash entre mosquitto_passwd 2.1.2 y mosquitto 2.0.21, y las restricciones de la política de ejecución de scripts de PowerShell. Estas dificultades, aunque no anticipadas en el diseño, constituyeron oportunidades de aplicar metodología de diagnóstico sistemático y documentar las soluciones en el archivo de planificación del proyecto.

Una limitación conocida del sistema es la ausencia de cifrado en tránsito (TLS). Las credenciales MQTT viajan en texto plano dentro de la red local. En la premisa de una red de laboratorio controlada, esta limitación es aceptable y está documentada explícitamente como decisión de alcance, no como omisión inadvertida.

8. Conclusiones

El presente trabajo logró diseñar, implementar y verificar un sistema de control para brazo robótico pick & place con autenticación en dos capas independientes, operando establemente en red de área local. Todos los requerimientos funcionales y no funcionales definidos en la sección 4 fueron satisfechos, como lo evidencian los 18 casos de prueba documentados.

Los aportes principales del trabajo son: (1) la aplicación sistemática de principios OWASP (mínimo privilegio, defensa en profundidad, fail-fast) en un contexto de sistema embebido académico; (2) la documentación de decisiones de arquitectura mediante ADR antes de implementar, lo que facilitó tanto el desarrollo como la defensa oral; (3) la separación de la autenticación de la aplicación (credenciales MQTT) de la autenticación del operador (JWT), estableciendo dos barreras independientes de seguridad.

Las principales dificultades encontradas fueron de compatibilidad de versiones de software en el entorno Windows de desarrollo. La resolución sistemática de cada problema, documentada en `planificacion.md`, generó un registro valioso del proceso de ingeniería real.

Como líneas de trabajo futuro se identifican:

- Implementación de TLS en el broker Mosquitto (puerto 8883) con certificados X.509, para cifrar las credenciales MQTT en tránsito.
- Migración de la gestión de usuarios a una base de datos relacional (SQLite o PostgreSQL), permitiendo alta y baja de operadores sin reiniciar el backend.
- Integración de una cinta transportadora para automatizar el suministro de cajas a la zona de recolección, eliminando la intervención manual de colocación y habilitando un ciclo productivo continuo.
- Implementación de refresh tokens JWT para sesiones de operación extendidas, con política de revocación por cierre de sesión explícito.
- Exploración de MQTT Sparkplug B como protocolo de alto nivel sobre MQTT para estandarizar el formato de los payloads de telemetría en un contexto de múltiples nodos IoT.

Referencias

- [1] F. Bevilacqua, "Sistema de Control v4.0 para Brazo Robótico Pick & Place", Trabajo Práctico, Ingeniería en Computación II, UNRAFA, Rafaela, 2025.
- [2] M. Collina, G. Corazza, A. Vanelli-Coralli y G. Corazza, "Introducing the MQTT Protocol for the Internet of Things", en Int. Symp. on Advanced Satellite Multimedia Systems, 2012.
- [3] A. Stanford-Clark e I. Nipper, "MQ Telemetry Transport (MQTT) V3.1 Protocol Specification", IBM DeveloperWorks, 2010. [Online]. Available: <https://public.dhe.ibm.com/software/dw/webservices/ws-mqtt/mqtt-v3r1.html>
- [4] OASIS Standard, "MQTT Version 3.1.1", OASIS, 2014. [Online]. Available: <https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html>
- [5] T. Yokotani y Y. Sasaki, "Comparison with HTTP and MQTT on Required Network Resources for IoT", en Proc. Int. Conf. on Control, Electronics, Renewable Energy and Communications (ICCEREC), 2016.
- [6] K. Schwab, La Cuarta Revolución Industrial. Barcelona: Debate, 2016.
- [7] OWASP Foundation, "OWASP IoT Top 10", 2018. [Online]. Available: <https://owasp.org/www-pdf-archive/OWASP-IoT-Top-10-2018-final.pdf>
- [8] S. Ramírez, "FastAPI", [Online]. Available: <https://fastapi.tiangolo.com/>
- [9] M. Jones, J. Bradley y N. Sakimura, "JSON Web Token (JWT)", RFC 7519, IETF, 2015. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc7519>
- [10] N. Provos y D. Mazières, "A Future-Adaptable Password Scheme", en Proc. USENIX Annual Technical Conference, 1999.
- [11] I. Sommerville, Software Engineering, 10ª ed. Boston: Pearson, 2015.
- [12] Eclipse Mosquitto, "An open source MQTT broker", [Online]. Available: <https://mosquitto.org/>

Anexo A — Instrucciones de ejecución del sistema

El sistema requiere los siguientes componentes de software instalados en la PC de operación: Mosquitto 2.0.21 (Windows x64), Python 3.11+, Thonny IDE 4.x y un servidor HTTP estático (Live Server de VSCode o `python -m http.server`).

Secuencia de arranque:

- 1. Ejecutar `ARRANCAR_SISTEMA.ps1` (script PowerShell incluido en la raíz del proyecto, ajustar ruta donde se encuentra el archivo). Inicia Mosquitto y el backend FastAPI en el orden correcto.
- 2. Conectar el ESP32 por USB-C y ejecutar el firmware en Thonny (F5). Verificar "MQTT conectado (intento #1)" en la consola.
- 3. Servir la GUI con un servidor HTTP local y abrir `http://localhost:5500/login.html` en el navegador.
- 4. Autenticarse con las credenciales de operador configuradas en `backend/.env`.

El repositorio del proyecto, la guía de instalación completa (`GUIA_INSTALACION.md`) y el archivo de planificación con decisiones de arquitectura (`planificacion.md`) se encuentran disponibles en el repositorio del proyecto entregado junto a este artículo.

Anexo B — Diagrama de conexiones eléctricas del hardware

El ESP32 NodeMCU se alimenta por USB-C desde la PC. Los cuatro servomotores SG90 se alimentan exclusivamente desde una fuente externa de 5V/2A; sus pines de señal se conectan a los GPIOs 25, 26, 27 y 32 del ESP32. El sensor KY-032 se alimenta desde el pin de 3.3V del ESP32. El pin OUT del sensor se conecta al GPIO 33 con pull-up interno habilitado por software (señal LOW ante detección).

Es crítico que el GND de la fuente externa de 5V, el GND del ESP32 y el GND del sensor KY-032 compartan un punto de referencia común. Sin GND común, las señales PWM son flotantes y el comportamiento de los servos es impredecible. Alimentar los servos desde el pin de 3.3V del ESP32 produce reset por brownout (código de causa 6) o daño permanente al regulador del microcontrolador.